

Stack vs Heap

Припомняме, че поменливите от текстов тип `string` са непроменяеми `{ get; }` – read-only.

Също така имаме 2 вида транслятори в C#, които прихващат грешки на различно ниво: първият е компилатора (индикира червено зигзагообразно подчертаване на грешката още докато пишем кода) и вторият е CLR – Runtime (при стартирането на проекта от Run или F5).

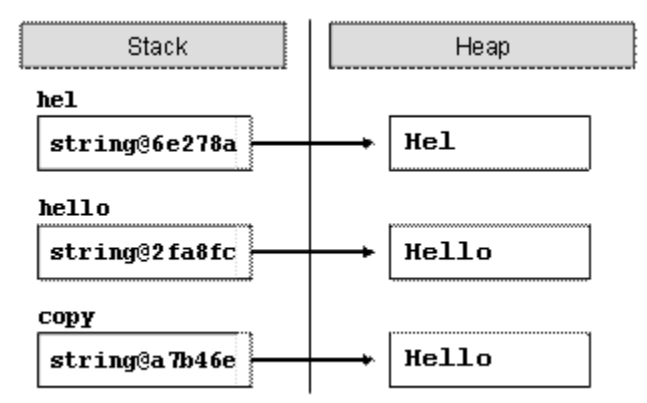
В C# управлението на паметта се осъществява чрез **Stack (стек/** по-скоро със значение на шпиндел) и **Heap (куп/купчина)**, които са основни части на CLR (Common Language Runtime).

Garbage Collector

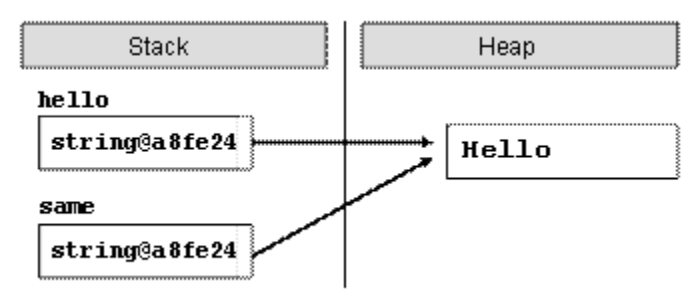
- Автоматично управлява паметта само в Heap
- Освобождава обекти, към които няма живи референции
- Стекът се освобождава автоматично **много преди GC** да се намеси, просто с преместване на указателя (референцията).

```
void Method()
{
    int x = 10;    // x е в Stack – GC никога не вижда това!

}                // x изчезва моментално при излизане от метода
```



Съществува вътрешна проверка, която спестява създаването на дублирани символни низове в паметта. Тази оптимизация се нарича **интерниране на низовете (strings interning)** и благодарение на нея две еднакви текстови променливи ще сочат към един и същ общ блок от паметта. Това намалява разхода на памет и оптимизира някои операции, например сравнението на два (и повече) напълно съвпадащи низа. Те се записват в паметта по следния начин:



Stack

- **Бърза** област от паметта (LIFO - Last In First Out - последен влязъл, първи излязъл) - като шпиндел за дискове.
- **С ограничен размер на паметта** (1 – 8 MB)
- **Автоматично** управление на паметта (освобождава се при излизане от обхвата {})
- **Съхранява:**
 - Стойности на типове struct като DateTime (ако не са част от обекти в Heap)
 - Локални променливи като int, също и параметри
 - Референцията винаги е в Stack, а самият обект в Heap (за string, масив[])



Откъде ни е познато? – **Stack Overflow** – Препълване на паметта.

Heap

- **По-бавна** област от паметта
- **Динамично** заделяне на памет
- **Garbage Collector (събирач на боклук)** автоматично освобождава неизползвани обекти
- **Съхранява:**
 - Обекти от класове (class)

- Масиви []
- Низове (strings)
- Големи структури (или полета на класове)

Примери:

```
class Person
{
    public string Name;           // Person е в Heap
}

Person p = new Person();         // референция 'p' в Stack, обект в Heap

void Method()
{
    int x = 10;                  // стойност в Stack
    DateTime date = DateTime.Now; // в Stack (защото е struct)

    string s = "Hello";          // референция 's' в Stack, обект в Heap
    int[] numbers = new int[10]; // референция в Stack, масив в Heap
}                                // при излизане от метода Stack се почиства
```

Зад 1: Напишете програма, която при въвеждане на коефициентите (**a**, **b** и **c**) на квадратно уравнение: ax^2+bx+c , изчислява и извежда неговите реални корени (ако има такива). Квадратните уравнения могат да имат 0, 1 или 2 реални корена.

Зад 2: Напишете програма, която намира най-голямото по стойност число измежду дадени 5 числа.